Boggle Algorithms: Pseudocode

This document describes the five algorithms currently implemented in `server.py`. Every algorithm receives the same:

- flattened 4x4 board;
- candidate dictionary words;
- map of adjacent cells.

The algorithms record one valid path for each discovered word. A cell cannot be reused within one path.

Shared data structures

Board indexing

```
index(row, column):
    return row * 4 + column
```

The 4x4 board is flattened into 16 cells:

```
board = [
    cell_0, cell_1, ..., cell_15
]
```

Neighbor map

```
build_neighbors():
    neighbors = empty map

    for row from 0 to 3:
        for column from 0 to 3:
            current = index(row, column)
            neighbors[current] = empty list

            for row_delta in [-1, 0, 1]:
                for column_delta in [-1, 0, 1]:
                    if row_delta == 0 and column_delta == 0:
                        continue

                    next_row = row + row_delta
                    next_column = column + column_delta

                    if next_row and next_column are inside the board:
                        add index(next_row, next_column)
                            to neighbors[current]

    return neighbors
```

Visited-cell bit mask

Each board cell has one bit in an integer.

```
is_used(mask, cell):
    return (mask AND (1 shifted left by cell)) is not zero

mark_used(mask, cell):
    return mask OR (1 shifted left by cell)
```

Candidate words

```
get_candidates(dictionary, minimum_length):
    candidates = empty set

    for word in dictionary:
        if minimum_length <= length(word) <= 16:
            add word to candidates

    return candidates
```

Convert a path to coordinates

```
to_coordinates(cells):
    coordinates = empty list

    for cell in cells:
        row = cell DIVIDED BY 4
        column = cell MODULO 4
        append (row, column) to coordinates

    return coordinates
```

Trie construction

The trie stores dictionary words character by character. A terminal node stores the complete word. The `left` value counts undiscovered words below a node and is used by subtree pruning.

```
create_trie(words):
    root = new TrieNode

    for word in words:
        node = root
        node.left = node.left + 1

        for character in word:
            if character is not a child of node:
                node.children[character] = new TrieNode

            node = node.children[character]
            node.left = node.left + 1

        node.word = word

    return root
```

1. Brute-force DFS

Brute force explores board paths without checking dictionary prefixes. It only checks whether a complete current string is a candidate word.

```
brute_force(board, candidates, neighbors):
    found = empty map
    states = 0
    checks = 0
    pruned = 0

    longest_word_length = maximum length in candidates
                          or 0 if candidates is empty

    search(cell, used_mask, path, text):
        states = states + 1

        if text is in candidates:
            if text is not already in found:
                found[text] = to_coordinates(path)

        if length(text) == longest_word_length:
            return

        for next_cell in neighbors[cell]:
            checks = checks + 1

            if is_used(used_mask, next_cell):
                pruned = pruned + 1
                continue

            search(
                next_cell,
                mark_used(used_mask, next_cell),
                path plus next_cell,
                text plus board[next_cell]
            )

    for start_cell from 0 to 15:
        search(
            start_cell,
            mark_used(0, start_cell),
```

```
            [start_cell],
            board[start_cell]
        )

    return found, states, checks, pruned
```

2. Prefix-set DFS

Prefix-set DFS creates a set of every prefix appearing in the candidate dictionary. A path is explored only while its current text is a known prefix.

```
prefix_dfs(board, candidates, neighbors):
    prefixes = empty set

    for word in candidates:
        for end_position from 1 to length(word):
            add word[0:end_position] to prefixes

    found = empty map
    states = 0
    checks = 0
    pruned = 0

    search(cell, used_mask, path, text):
        states = states + 1

        if text is in candidates:
            if text is not already in found:
                found[text] = to_coordinates(path)

        for next_cell in neighbors[cell]:
            checks = checks + 1

            if is_used(used_mask, next_cell):
                pruned = pruned + 1
                continue

            next_text = text plus board[next_cell]

            if next_text is not in prefixes:
                pruned = pruned + 1
                continue

            search(
                next_cell,
                mark_used(used_mask, next_cell),
                path plus next_cell,
                next_text
```

```
            )

    for start_cell from 0 to 15:
        start_text = board[start_cell]

        if start_text is in prefixes:
            search(
                start_cell,
                mark_used(0, start_cell),
                [start_cell],
                start_text
            )
        else:
            pruned = pruned + 1

    return found, states, checks, pruned
```

3. Trie DFS

Trie DFS follows trie child nodes instead of creating and looking up complete prefix strings.

```
trie_dfs(board, candidates, neighbors):
    trie = create_trie(candidates)
    found = empty map
    states = 0
    checks = 0
    pruned = 0

    search(cell, used_mask, path, node):
        states = states + 1

        if node.word is not empty:
            if node.word is not already in found:
                found[node.word] = to_coordinates(path)

        for next_cell in neighbors[cell]:
            checks = checks + 1

            if is_used(used_mask, next_cell):
                pruned = pruned + 1
                continue

            character = board[next_cell]
            child = node.children[character]

            if child does not exist:
                pruned = pruned + 1
                continue

            search(
                next_cell,
                mark_used(used_mask, next_cell),
                path plus next_cell,
                child
            )

    for start_cell from 0 to 15:
        character = board[start_cell]
```

```
        child = trie.children[character]

        if child exists:
            search(
                start_cell,
                mark_used(0, start_cell),
                [start_cell],
                child
            )
        else:
            pruned = pruned + 1

    return found, states, checks, pruned
```

4. Trie DFS with subtree pruning

This algorithm extends Trie DFS with the `left` count. When a word is found, its trie path is decremented. A node with `left == 0` has no undiscovered words below it and can be skipped.

```
subtree_pruned_dfs(board, candidates, neighbors):
    trie = create_trie(candidates)
    found = empty map
    states = 0
    checks = 0
    pruned = 0

    remove_word(word):
        node = trie
        node.left = node.left - 1

        for character in word:
            node = node.children[character]
            node.left = node.left - 1

    search(cell, used_mask, path, node):
        states = states + 1

        if node.word is not empty:
            found[node.word] = to_coordinates(path)
            remove_word(node.word)
            node.word = empty

        if node.left == 0:
            return

        for next_cell in neighbors[cell]:
            checks = checks + 1

            child = node.children[board[next_cell]]

            if is_used(used_mask, next_cell):
                pruned = pruned + 1
                continue

            if child does not exist:
                pruned = pruned + 1
```

```
                continue

            if child.left == 0:
                pruned = pruned + 1
                continue

            search(
                next_cell,
                mark_used(used_mask, next_cell),
                path plus next_cell,
                child
            )

    for start_cell from 0 to 15:
        child = trie.children[board[start_cell]]

        if child exists and child.left > 0:
            search(
                start_cell,
                mark_used(0, start_cell),
                [start_cell],
                child
            )
        else:
            pruned = pruned + 1

    return found, states, checks, pruned
```

5. Dynamic programming with memoization

Dynamic programming stores states that have already been expanded. The state must include:

```
(current cell, used-cell mask, current trie node)
```

The current cell alone is not enough because different used masks can leave different legal moves.

```
dynamic_programming(board, candidates, neighbors):
    trie = create_trie(candidates)
    found = empty map
    visited_states = empty set
    states = 0
    checks = 0
    pruned = 0

    search(cell, used_mask, path, node):
        state = (cell, used_mask, identity(node))

        if state is in visited_states:
            pruned = pruned + 1
            return

        add state to visited_states
        states = states + 1

        if node.word is not empty:
            if node.word is not already in found:
                found[node.word] = to_coordinates(path)

        for next_cell in neighbors[cell]:
            checks = checks + 1

            if is_used(used_mask, next_cell):
                pruned = pruned + 1
                continue

            child = node.children[board[next_cell]]

            if child does not exist:
                pruned = pruned + 1
                continue

            search(
                next_cell,
```

```
                mark_used(used_mask, next_cell),
                path plus next_cell,
                child
            )

    for start_cell from 0 to 15:
        child = trie.children[board[start_cell]]

        if child exists:
            search(
                start_cell,
                mark_used(0, start_cell),
                [start_cell],
                child
            )
        else:
            pruned = pruned + 1

    return found, states, checks, pruned
```

Shared benchmark pseudocode

The benchmark runs every algorithm with the same prepared input:

```
benchmark(board, dictionary, minimum_length):
    algorithms = [
        ("Brute-force DFS", brute_force),
        ("Prefix-set DFS", prefix_dfs),
        ("Trie DFS", trie_dfs),
        ("Trie DFS with subtree pruning", subtree_pruned_dfs),
        ("Dynamic programming", dynamic_programming),
    ]

    candidates = get_candidates(dictionary, minimum_length)
    chars = flatten(board)
    adjacent = build_neighbors()
    results = empty list

    for name, algorithm in algorithms:
        start preparation timer
        prepare algorithm-specific data if needed
        stop preparation timer

        start search timer
        words, states, checks, pruned = algorithm(
            chars,
            candidates,
            adjacent
        )
        stop search timer

        append Result(
            name,
            words,
            preparation time,
            search time,
            states,
            checks,
            pruned
        ) to results
```

```
    return results
```

Correctness requirement

All five algorithms should return the same set of words for the same board, dictionary, and minimum length. They may discover different valid paths for a word, but every returned path must:

1. use adjacent cells;
2. avoid reusing a cell;
3. spell the returned word;
4. satisfy the selected length range;
5. contain a word from the candidate dictionary.